

# Tarea Final de la Unidad PROG06

## Gestión de Biblioteca con Ficheros, CLI y GUI

### Entrega

Deberás crear una carpeta comprimida en formato **ZIP** con el nombre:

**Apellido\_Nombre\_Tarea\_Final\_PROG06**

Asegúrate que el nombre no contenga la letra ñ, tildes ni caracteres especiales extraños. Así por ejemplo la alumna **Begoña Sánchez Mañas para la segunda unidad del MP de MIA**, debería nombrar esta tarea como:

**sanchez\_manas\_begona\_Tarea\_Final\_PROG06.zip**

Dentro de esta carpeta incluye todas las clases necesarias para la ejecución del programa. Finalmente, sube el archivo comprimido en la tarea correspondiente del aula virtual.

### Enunciado del Problema

#### Título:

Gestión de Biblioteca con persistencia en fichero e interfaz gráfica.

#### Descripción

Desarrolla un programa en Java que permita gestionar una colección de libros utilizando programación orientada a objetos.

El programa deberá permitir administrar los libros mediante dos interfaces diferentes:

- Una aplicación en **línea de comandos (CLI)**.
- Una aplicación con **interfaz gráfica (GUI)** desarrollada utilizando **Swing** y **AWT**.

Los datos de los libros deberán almacenarse en un fichero de texto, permitiendo que la información se conserve entre ejecuciones del programa.

El sistema deberá leer automáticamente los datos al iniciar y permitir guardarlos en fichero durante la ejecución.

## **Especificaciones**

### **Clase Libro**

La clase Libro debe contener al menos los siguientes atributos:

- String isbn: código identificador único del libro.
- String titulo
- String autor
- boolean disponible

Funciones mínimas:

- Constructor completo.
- Métodos getters y setters.
- Método prestar() (Cambia el estado de disponible).
- Método devolver() (Cambia el estado de disponible).
- Método toString() para visualización.

### **Clase Biblioteca**

Gestionará la colección de libros mediante un ArrayList.

Si se considera necesario, se puede crear un HashMap para facilitar las búsquedas por ISBN.

Debe incluir al menos:

- Añadir libro (sin permitir ISBN duplicados).
- Buscar libro por ISBN.
- Prestar libro. Busca el libro por ISBN (usa el método anterior), si lo encuentra y está disponible llama al método prestar del libro.
- Devolver libro. Como el anterior, pero llama al método devolver de libro.
- Mostrar listado de libros.

### **Clase FicheroLibros**

Formato del fichero:

Cada libro ocupará una línea con el siguiente formato:

titulo;autor;isbn;disponible

Ejemplo:

El Quijote;Cervantes;9788491050295;true

Clase utilitaria con métodos estáticos encargada de la persistencia de datos.

Debe utilizar obligatoriamente:

- FileReader y BufferedReader para lectura.
- FileWriter y BufferedWriter para escritura.

**Método: public static void leerFichero(String f, Biblioteca b)**

### Objetivo

Leer el fichero línea a línea y cargar los libros en la biblioteca.

### Pasos recomendados (guía)

1. Abrir el fichero con:
  - `FileReader fr = new FileReader(f);`
  - `BufferedReader br = new BufferedReader(fr);`
2. Leer líneas mientras exista una nueva línea:
  - `while ((linea = br.readLine()) != null)`
3. Limpiar la línea:
  - `linea = linea.trim();`
  - Si está vacía, se ignora: `if (linea.isEmpty()) continue;`
4. Separar los campos:
  - `String[] partes = linea.split(";");`
  - Si no hay 4 partes (formato incorrecto), se ignora la línea.
5. Extraer y limpiar cada dato:
  - `String titulo = partes[0].trim();`
  - `String autor = partes[1].trim();`
  - `String isbn = partes[2].trim();`
  - `String dispStr = partes[3].trim();`

Importante: todo lo leído del fichero es **String**, por tanto si algún dato no es String (por ejemplo un boolean), hay que convertirlo.

6. Convertir el boolean:

- `boolean disponible = Boolean.parseBoolean(dispNet);`  
Nota: se recomienda comprobar antes que `dispNet` sea "true" o "false" para evitar errores silenciosos.

#### 7. Validaciones (ejemplos):

- Título, autor e ISBN no vacíos.
- El campo disponible debe ser true o false.

#### 8. Crear el libro y añadirlo a la biblioteca:

- `boolean ok = b.altaLibro(new Libro(isbn, titulo, autor, disponible));`

La inserción debe hacerse a través de `altaLibro` para:

- evitar ISBN duplicados
- reutilizar la lógica de validación de la biblioteca

#### 9. Manejo de errores obligatorio:

- `FileNotFoundException`: si el fichero no existe, el programa no debe fallar. Se mostrará:  
“No existe el fichero de datos. Se creará al guardar.”
- `IOException`: mostrar un mensaje informativo de error.  
“Error de entrada/salida:...”

**Método: `public static void escribirFichero(String f, Biblioteca b)`**

#### **Objetivo**

Guardar en el fichero todos los libros que existen en la biblioteca, usando el mismo formato que se utiliza para leer.

#### **Pasos recomendados (guía)**

1. Abrir el fichero con:
  - `FileWriter fw = new FileWriter(f);`
  - `BufferedWriter bw = new BufferedWriter(fw);`
2. Recorrer la colección de libros:
  - `for (Libro l : b.getLibros())`
3. Construir una línea con el formato exacto:
  - `titulo;autor;isbn;disponible`

Ejemplo de construcción:

- `String linea = l.getTitulo() + ";" + l.getAutor() + ";" + l.getIsbn() + ";" + l.isDisponible();`
- 4. Escribir la línea y salto de línea:
  - `bw.write(linea);`
  - `bw.newLine();`
- 5. Al terminar, mostrar mensaje de confirmación:
  - “Fichero guardado correctamente.”
- 6. Manejo de excepciones:
  - Capturar `IOException` y mostrar mensaje informativo:  
“Error al escribir el fichero: ...”

## **Programas Principales**

### **1. Aplicación en línea de comandos (MainCLI)**

Debe incluir un menú repetitivo con las siguientes opciones:

1. Añadir libro
2. Mostrar libros
3. Prestar libro
4. Devolver libro
5. Buscar libro por ISBN
6. Guardar datos en fichero
7. Salir

### **2. Aplicación con Interfaz Gráfica (MainGUI)**

La interfaz debe desarrollarse utilizando **Swing** y **AWT**.

Componentes mínimos:

- `JFrame` como ventana principal.
- `JPanel` con distintos layouts (`BorderLayout`, `GridLayout` o `FlowLayout`).
- `TextField` para introducción de datos.
- `Button` para acciones.
- `JTable` o `JList` para mostrar los libros.

- JOptionPane para mensajes y errores.

Funcionalidades obligatorias:

1. Añadir libro
  - Introducción de datos mediante formulario.
  - Validación de ISBN no repetido.
2. Mostrar libros
  - Visualización actualizada de la colección.
3. Prestar libro
  - Cambia el estado del libro a no disponible.
4. Devolver libro
  - Cambia el estado a disponible.
5. Buscar libro.
  - Buscar libro por ISBN.
6. Guardar datos
  - Escritura del fichero mediante la clase FicheroLibros.

### **Requisitos**

- El programa intentará cargar automáticamente los libros desde el fichero utilizando la clase FicheroLibros al iniciar la ejecución.
- Si el fichero existe, los libros se cargarán correctamente en la biblioteca.
- Si el fichero no existe, se deberá capturar la excepción FileNotFoundException en el programa principal y la aplicación continuará con la biblioteca vacía, mostrando el siguiente mensaje informativo:  
“No existe fichero de datos. Se creará al guardar.”

### **Persistencia de Datos**

- Los libros se almacenarán en un fichero de texto.
- Cada libro ocupará una línea.
- El programa deberá poder leer y escribir correctamente el fichero.
- Los errores de lectura o escritura deberán gestionarse mediante excepciones.

### Rúbrica de Evaluación

| Criterio                          | Descripción                                     | Puntos           |
|-----------------------------------|-------------------------------------------------|------------------|
| Diseño POO                        | Correcta implementación de Libro y Biblioteca   | 2                |
| Lectura y escritura de ficheros   | Uso correcto de BufferedReader y BufferedWriter | 3                |
| Interfaz CLI                      | Funcionamiento correcto del menú y validaciones | 1                |
| Interfaz gráfica Swing            | Uso adecuado de componentes y eventos           | 2                |
| Validaciones y control de errores | Manejo de excepciones y mensajes al usuario     | 1                |
| Organización y buenas prácticas   | Código estructurado y legible                   | 1                |
| <b>Total</b>                      |                                                 | <b>10 puntos</b> |

### Ayuda para la creación de la clase principal MainGUI:

La clase extiende JFrame:

- `public class MainGUI extends JFrame`

Esto permite que la clase sea una ventana principal de la aplicación.

### Atributos básicos

- Una instancia de Biblioteca para gestionar los libros.
- Campos de entrada con JTextField (título, autor, ISBN).
- Una tabla JTable para mostrar los libros.
- Un DefaultTableModel para controlar los datos mostrados en la tabla.

Ejemplo de atributos:

- `private Biblioteca biblioteca;`
- `private JTextField tfTitulo, tfAutor, tfIsbn;`
- `private JTable tabla;`
- `private DefaultTableModel modeloTabla;`

## Constructor: carga y configuración inicial

En el constructor se recomienda:

1. Crear la biblioteca:
  - `biblioteca = new Biblioteca();`
2. Cargar automáticamente el fichero al iniciar:
  - `FicheroLibros.leerFichero(FICHERO, biblioteca);`
3. Construir la interfaz:
  - `construirInterfaz();`
4. Mostrar los datos en la tabla:
  - `refrescarTabla();`
5. Configurar ventana:
  - tamaño (`setSize`)
  - centrado (`setLocationRelativeTo(null)`)
  - cierre (`setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE)`)
  - visible (`setVisible(true)`)

## Construcción de la interfaz (`construirInterfaz()`)

Se recomienda usar un BorderLayout (`setLayout(new BorderLayout(10, 10));`) en el JFrame y repartir la pantalla en 3 zonas:

### Panel superior (NORTH): formulario

- Crear un JPanel con GridLayout(3, 2)  
`JPanel panelFormulario = new JPanel(new GridLayout(3,2,8,8));`
- Añadir JLabel + JTextField para:
  - Título
  - Autor
  - ISBN`add(panelFormulario, BorderLayout.NORTH);`

### Panel central (CENTER): tabla

- Crear DefaultTableModel con columnas: `String[] columnas = {"Título", "Autor", "ISBN", "Disponible"};`



- La tabla debe ser de solo lectura: sobrescribir `isCellEditable()` y devolver `false`:

```
modeloTabla = new DefaultTableModel(columnas, 0) {
    @Override
    public boolean isCellEditable(int row, int column) {
        return false; // tabla solo lectura
    }
};
```

- Crear `JTable` con ese modelo: `tabla = new JTable(modeloTabla);`
- Añadir la tabla dentro de un `JScrollPane`: `JScrollPane scroll = new JScrollPane(tabla);`
- Colocar en la parte central: `add(scroll, BorderLayout.CENTER);`

### Panel inferior (SOUTH): botones

- Crear un `JPanel` con `FlowLayout`: `JPanel panelBotones = new JPanel(new FlowLayout(FlowLayout.CENTER, 10, 10));`
- Botones mínimos:
  - Añadir
  - Mostrar
  - Prestar
  - Devolver
  - Buscar por ISBN
  - Guardar
- Colocar en la parte inferior: `add(panelBotones, BorderLayout.SOUTH);`

### Eventos

A cada botón se le asigna un `ActionListener` que llama a su método correspondiente:

- `btnAnadir.addActionListener(e -> accionAnadir());`
- `btnGuardar.addActionListener(e -> accionGuardar());`
- etc.

### Acciones (métodos de los botones)

### **accionAnadir()**

Debe:

1. Leer los valores de los JTextField.
2. Validar datos:
  - no vacíos
3. Crear el libro y añadirlo usando biblioteca.altaLibro(libro)
4. Mostrar mensaje con JOptionPane
5. Limpiar formulario y refrescar tabla

### **accionMostrar()**

Simplemente llama:

- refrescarTabla();

### **accionPrestar() y accionDevolver()**

Se recomienda:

1. Obtener ISBN.
2. Validar ISBN.
3. Llamar a:
  - biblioteca.prestar(isbn) o
  - biblioteca.devolver(isbn)
4. Mostrar resultado con JOptionPane
5. Refrescar tabla

### **accionBuscar()**

Debe:

1. Pedir ISBN por JOptionPane.showInputDialog
2. Validar ISBN
3. Buscar: biblioteca.buscarPorIsbn(isbn)
4. Mostrar datos del libro si existe (título, autor, ISBN, disponible)

### **accionGuardar()**

Debe llamar a:

- FicheroLibros.escribirFichero(FICHERO, biblioteca);

## Utilidades importantes

### refrescarTabla()

Debe:

1. Vaciar el modelo.
2. Recorrer la biblioteca.
3. Añadir una fila por cada libro.

```
// Vaciar tabla
modeloTabla.setRowCount(0);

//Recorrer biblioteca
for (Libro l : biblioteca.getLibros()) {
    Object[] fila = {
        l.getTitulo(),
        l.getAutor(),
        l.getIsbn(),
        l.isDisponible() ? "Sí" : "No"
    };
    //añadir una fila por cada libro
    modeloTabla.addRow(fila);
}
```

### limpiarFormulario()

Vacía campos y devuelve el foco a título:

- tfTitulo.setText("");
- tfAutor.setText("");
- tfIsbn.setText("");
- tfTitulo.requestFocus();

## Ejecución correcta de Swing

La interfaz puede lanzarse con:

- SwingUtilities.invokeLater(MainGUI::new);

Evitando problemas.